Module 1 · Study notes

MICROCONTROLLERS AND EMBEDDED SYSTEMS • PCEOT501 • S5

From embedded systems to the 8051 hardware

A syllabus-aligned guide built from **Module1-ES+8051.pdf**, the 75-slide lecture deck by Dr. Mayadevi N, CET. Image-only syllabus tables, architecture drawings, register maps and worked examples have been read visually and incorporated.

What Module 1 actually requires

The syllabus on slides 4 and 33 allocates **9 contact hours**: introduction to microprocessors and microcontrollers; their comparison; embedded-system application domains, features and characteristics; hard and soft real-time systems; 8051 architecture, I/O port structure, register organization, general-purpose RAM, bit-addressable RAM and SFRs.

Study route	Pages	What you should be able to do
Foundations	2-6	Explain an embedded control loop, classify deadlines and compare processors.
8051 hardware	7-10	Draw the architecture and explain the CPU registers and PSW.
Memory organization	11-15	Draw the memory map; solve bank, stack and bit-address problems.
Pins and ports	16-19	Explain pin functions, port circuitry and external-memory signals.
Exam preparation	20-23	Practise short and long answers; use the final revision sheet.
Source record	24	Locate the slide evidence and supplementary technical references.

How to use this guide

Read the explanations once, then redraw the diagrams and attempt the worked examples without looking at the answers. Pages 20-23 are for recall and revision. Small assembly fragments illustrate hardware behavior; full assembly/C programming, timer programming and peripheral interfacing belong to later modules.

Conventions: H means hexadecimal; # before a value means an immediate constant in assembly. A signal suffix # (for example, RD#) means active low. All specifications refer to the classic 8051 unless a variant is explicitly named. Diagrams are newly drawn, searchable vector artwork.

1 · What is an embedded system?

Lecture slides 13-14 and 35-37

An **embedded system** is an electronic computing system designed to perform a defined function or a limited set of functions within a product or larger system. It combines **hardware** with **software or firmware** that implements the application.

The key idea is **dedicated purpose**. A washing-machine controller repeatedly measures inputs and controls a wash sequence; a desktop computer is intended to run many unrelated applications. An embedded product can still have a display, user interaction, networking or several software tasks.

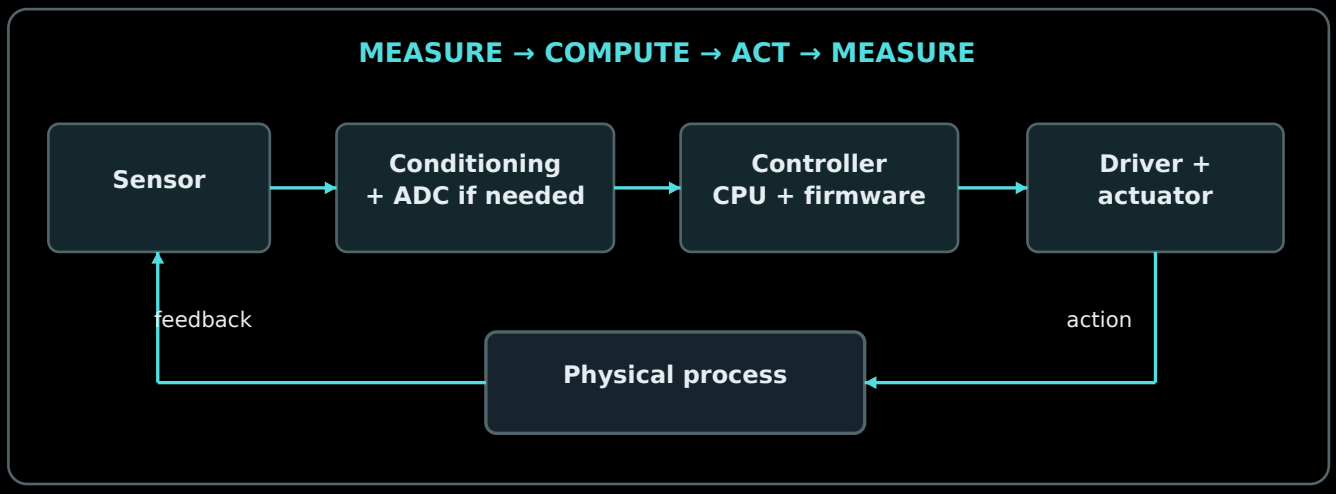


Figure 1. A typical embedded control loop. Some systems use digital sensors directly and need no external ADC.

Block	Role in the system	Temperature-control example
Sensor / input	Converts a physical condition into an electrical signal.	Temperature sensor measures the room.
Input interface	Conditions, protects or digitizes the signal.	Amplifier/filter and ADC, if the sensor is analog.
Controller	Executes firmware; makes decisions using inputs and stored state.	Compares measured temperature with the set point.
Output interface	Provides the voltage/current needed by the load.	Transistor, relay driver or motor driver.
Actuator / output	Changes the physical process; indicators report status.	Heater or fan changes the temperature.

Firmware is application code stored in nonvolatile program memory; working variables use RAM. A small system can run a repeated loop without an operating system, while a larger one may use an embedded OS.

Exam-ready definition

An embedded system is a hardware-software system dedicated to a specific application, usually forming part of a larger system, with constraints such as cost, power, memory, size and response time.

2 · Applications and characteristics

Lecture slides 15-22

Domain	Representative examples	What the controller does
Automotive	Engine control, ABS, airbags, cruise control	Reads sensors and controls an engine, brakes or safety mechanism.
Medical	Pacemaker, infusion pump, patient monitor	Measures physiological signals or delivers controlled treatment.
Home / buildings	Washing machine, microwave, HVAC, elevator	Sequences operations and maintains a controlled condition.
Industrial / robotics	Robot, process controller, instrumentation	Coordinates motion, measures signals and controls equipment.
Office / consumer	Printer, camera, television, network device	Controls a device-specific data and hardware workflow.
Transport / space	Traffic signalling, avionics, satellite	Responds to events while meeting reliability and timing needs.
Banking / agriculture	ATM, automated feeding or irrigation	Runs a defined transaction or field-control sequence.

The three core characteristics

• **Dedicated function:** the product has a defined application. “Single-functioned” in the slides means purpose-specific; it does not require exactly one task, one routine or one unchanging program.

• **Tightly constrained:** cost, energy, memory, physical size, processing speed and environmental conditions restrict design choices. Increasing one resource often affects another.

• **Reactive, often real-time:** the system responds to changes in inputs or the environment. Where a deadline is specified, the correct result must also arrive at the correct time.

Other useful characteristics

Reliability means providing the required service consistently. **Deterministic timing** makes response delays predictable. **Environmental suitability** includes temperature, vibration and electrical noise. **Maintainability and security** matter when firmware is updated or the device communicates with other systems.

Classify by the requirement, not just the product name

One car can contain several embedded systems: airbag deployment has a strict safety deadline; infotainment tolerates some delays. Likewise, a medical display and a life-critical controller can have very different timing requirements.

3 · Design trade-offs and comparisons

Lecture slides 22-25 and 43

Typical design objectives

Low power extends battery life and reduces heating. **Small code size** fits limited program memory. **Small physical size** fits the product enclosure. **Fast enough, predictable response** satisfies the application. **Low total cost** includes the chip, peripherals, PCB, power supply, development effort and support.

Aspect	Embedded system	General-purpose computer
Purpose	Known application or limited function set	Many user-selected applications
Resources	Chosen to meet product constraints	Often larger and more expandable
Timing	May have specified response deadlines	Usually optimized for throughput / user experience
Power / size	Often central constraints	Depend on the product; still important
Software	Firmware tied to product behavior	Applications frequently added or changed
Reliability	Failure consequence depends on application	Also required; criticality depends on use

The power-price-performance problem

A faster processor may shorten computation time, but can raise cost or power consumption. More memory supports richer software, but adds cost and possibly board area. A designer chooses sufficient resources with margin, rather than simply maximizing every specification.

Example: for a battery temperature logger, a controller that sleeps between readings may be a better fit than a powerful processor running continuously. For a motor-control loop, worst-case response time and the available I/O may matter more than a large program memory.

Three criteria for selecting a microcontroller

• **Task fit and cost:** processing speed, RAM/ROM capacity, I/O pins, timers, package, power and room for future changes.

• **Development support:** compiler, assembler, debugger, simulator, documentation and technical support.

• **Availability:** reliable supply, support lifetime and suitable alternatives.

Slide context: multicore processing, embedded operating systems, AI/ML, connectivity and device security are listed as trends. These motivate design choices but are not a separate detailed topic in the stated Module 1 syllabus. No historical market-share claim is treated as current.

4 · Hard, firm and soft real-time

Lecture slides 26-30; firm real-time is a useful supporting distinction

A **real-time system** is one whose correctness depends on both the computed result and the time at which the result is delivered. **Logical correctness** asks whether the answer is right; **temporal correctness** asks whether it arrives within the required time.

A job has a release time and a **deadline**, the latest acceptable completion time. For a job released at 0 ms with a deadline at 5 ms, completion at 4 ms meets the deadline; completion at 6 ms misses it, even if the calculation is accurate.

Type	Meaning of a missed deadline	Example under that requirement
Hard	A miss is a violation of the required system behavior; it may cause a serious or unsafe failure.	Safety-critical flight control or airbag actuation.
Firm	A late result has no useful value and is discarded; an occasional miss may be tolerated at system level.	A sensor sample that is obsolete after its control interval.
Soft	A late result can retain value, but service quality declines.	Audio/video playback or a noncritical user interface.

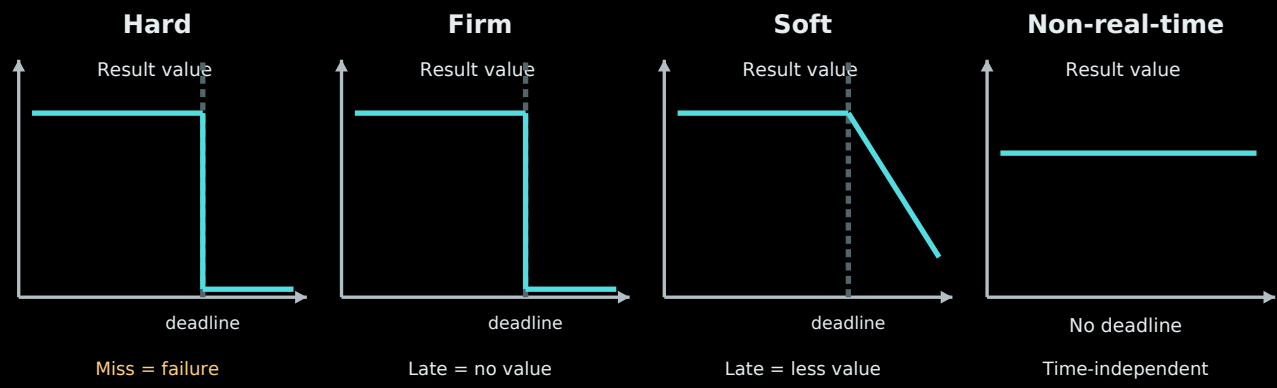


Figure 2. Conceptual result-value curves. The hard case also imposes a correctness requirement; these are not probability or speed plots.

Real-time does not simply mean “very fast”

A fast average response cannot guarantee a hard deadline. Worst-case computation, interrupt delay, communication and actuator delay all contribute to the end-to-end response. Predictable bounded delay matters more than an impressive average speed.

Useful answer structure

Define logical and temporal correctness; define the deadline; state the consequence of a miss; give an example. Do not claim that every embedded system is real-time or that every hard deadline miss necessarily causes a fatality.

5 · Microprocessor versus microcontroller

Lecture slides 36-40; memory organization also checked against [R3]

A **microprocessor** is primarily a programmable CPU: it fetches, decodes and executes instructions. A **microcontroller** integrates the CPU with memory and I/O peripherals on one chip, making a compact control system.

Aspect	Traditional microprocessor system	Microcontroller system
Integration	CPU chip with external memory and peripheral devices	CPU + program memory + RAM + ports + peripherals
Typical use	Flexible computing and larger software systems	Dedicated control and device functions
Memory / I/O	Chosen and expanded at system level	A specified set is integrated; some devices permit expansion
Board / cost	More external devices may be needed	Often fewer devices for a small control application
Power	Depends on CPU and complete system	Often designed with low-power operating modes

These are architectural tendencies, not absolute rules. Modern processors can integrate memory and peripherals; a microcontroller can run multiple tasks. The classic 8051 includes timers and a UART, but **does not include an on-chip ADC or DAC**.

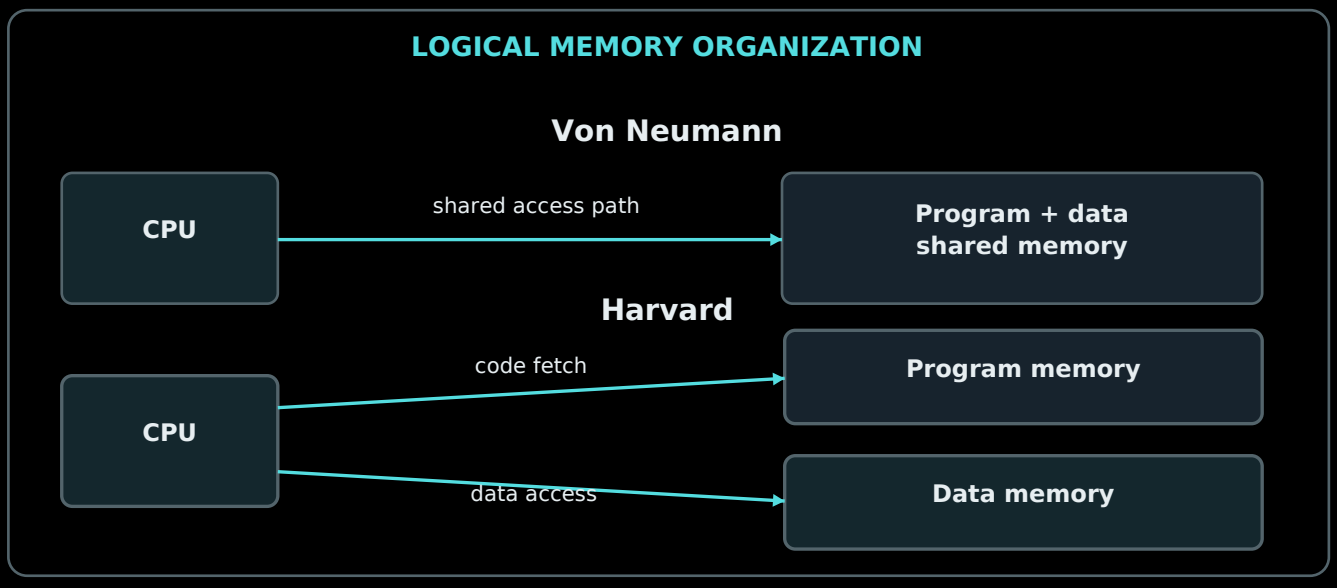


Figure 3. Von Neumann and Harvard organizations. The classic 8051 has separate logical code and data spaces, even though external transfers reuse port pins.

Remember the distinction

Microprocessor versus microcontroller describes **what is integrated**. Von Neumann versus Harvard describes **how code and data memory are organized**. These are different classifications.

6 · Classic 8051: the essential facts

Lecture slides 44-51; interrupt-count terminology clarified using [R1]

The 8051 is an **8-bit microcontroller** in Intel's MCS-51 family. Its normal data-processing width is 8 bits; that does not prevent it from using 16-bit address registers or performing multibyte calculations through multiple operations.

Feature	Classic 8051 specification
CPU / data bus	8-bit
PC and DPTR	16-bit each
PSW and stack pointer	8-bit each
On-chip program ROM	4 KB (4096 bytes)
On-chip data RAM	128 bytes
Program address space	64 KB: 0000H-FFFFH
External data address space	64 KB: 0000H-FFFFH, separate from code space
Parallel I/O	32 lines: four 8-bit ports, P0-P3
Timers / counters	Two 16-bit units, Timer 0 and Timer 1
Serial interface	One full-duplex serial port (UART)
Interrupt structure	6 request sources sharing 5 interrupt vectors
Other blocks	Oscillator/timing, interrupt control and external-memory bus control

Compare the basic family members

Feature	8051	8052	8031
On-chip program ROM	4 KB	8 KB	None
Internal RAM	128 B	256 B	128 B
Timers	2	3	2
I/O lines / serial ports	32 / 1	32 / 1	32 / 1

The **8031** requires external program memory. The **8052** adds RAM and a timer. Other compatible derivatives may add flash, ADCs or faster cores; do not substitute their specifications for the basic 8051.

The two "64 KB" spaces are not on-chip storage

A 16-bit address provides 2^{16} = 65,536 byte locations. Code and external data use distinct spaces, so code address 1234H and external-data address 1234H can identify different bytes.

Five vector groups: external 0, timer 0, external 1, timer 1 and serial. Serial RI and TI are separate requests sharing one vector, giving the slide's count of six sources.

7 · Understand the 8051 architecture

Lecture slides 47-54, including the image-only architecture on slide 49

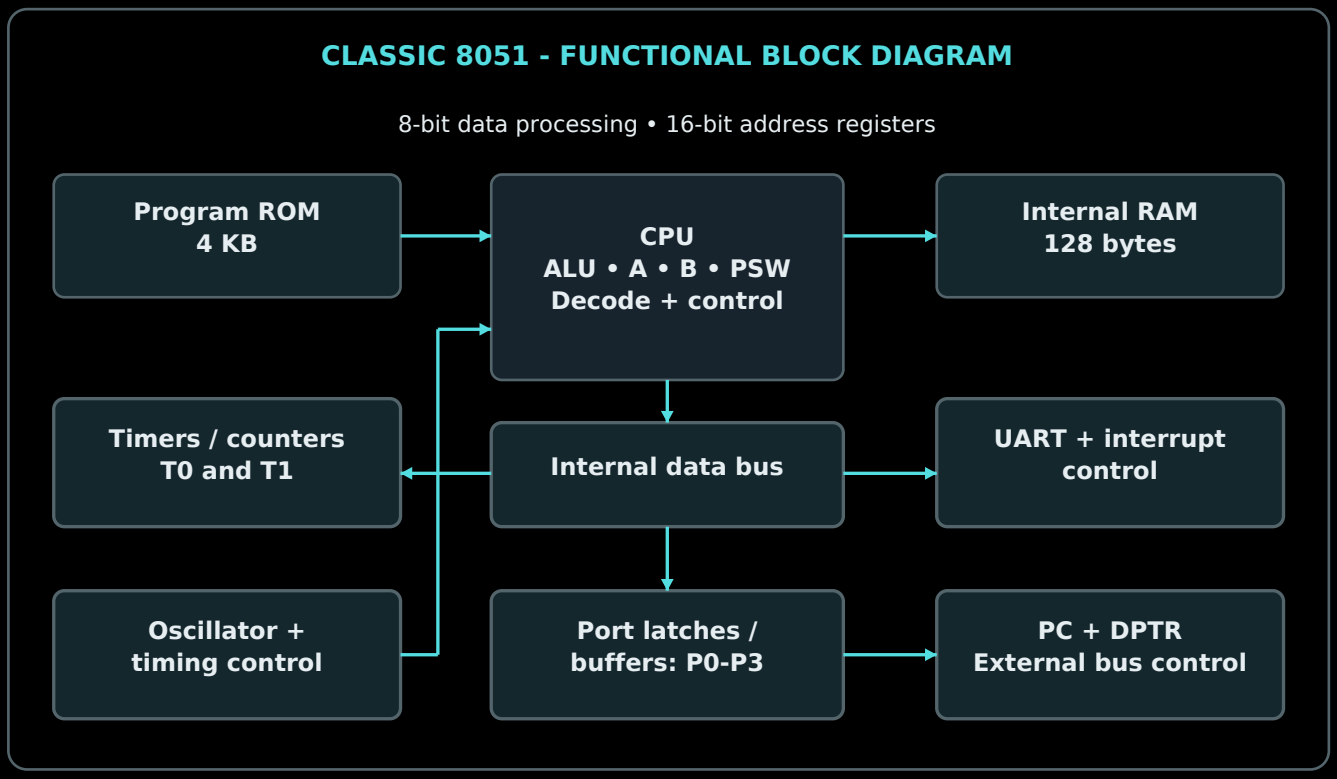


Figure 4. Redrawn functional architecture. Temporary registers and transistor-level details are omitted so the main paths remain clear.

What each part contributes

• **CPU:** the ALU performs arithmetic and logic; A and B hold important operands/results; the PSW records status and selects the working register bank.

• **Program path:** the program counter supplies an instruction address. Program memory supplies the instruction, which is decoded by the instruction decoder and executed under control signals.

• **Data path:** internal RAM holds register banks, bit variables, ordinary variables and the stack. SFRs provide software access to CPU and peripheral hardware.

• **Peripherals:** ports connect to external signals; timers measure intervals or count events; the serial block sends/receives data; interrupt control handles service requests.

• **Timing and bus control:** the oscillator provides the clock; timing/control coordinates execution. PC, DPTR and bus-control logic support memory addressing.

Fetch → decode → execute

The CPU fetches the next instruction, decodes the operation and operands, performs the specified action, and updates state for the next instruction. A branch changes the normal sequential instruction flow. The internal instruction register is a hardware implementation element, not an ordinary programmer-accessible register.

For a long architecture answer, draw Figure 4, state the key specifications and explain the function of each block. Timers, serial and interrupts are identified here; detailed programming belongs to later modules.

8 · CPU register organization

Lecture slides 50 and 55-57; [R2] confirms programmer-visible register widths

Register	Width / address	Main purpose
A / ACC	8 bits; E0H	Accumulator used by many arithmetic, logic and transfer instructions.
B	8 bits; F0H	Second operand/result register for MUL AB and DIV AB; can also hold temporary data.
R0-R7	8 bits each; selected RAM bank	Fast working registers. Their physical locations depend on RS1 and RS0.
DPTR	16 bits: DPH=83H, DPL=82H	Address pointer for external data and code-table access.
PC	16 bits; not an SFR	Tracks the instruction address; reset starts execution at 0000H.
SP	8 bits; 81H	Points to the top of the internal-RAM stack.
PSW	8 bits; D0H	Arithmetic status flags, bank-selection bits and software flag.

A and B in multiplication and division

MUL AB: two unsigned 8-bit operands produce a 16-bit result. A holds the low byte; B holds the high byte. Example: A=20H (32), B=10H (16); product=0200H, so A=00H and B=02H.

DIV AB: A is the unsigned dividend and B is the divisor. A receives the quotient and B receives the remainder. Example: 23 decimal divided by 5 gives A=4 and B=3. Division by zero is invalid and sets the overflow indication.

DPTR is a pair of bytes

Build a 16-bit address

DPTR = (DPH × 256) + DPL.
For DPTR = 3A7CH, DPH = 3AH and DPL = 7CH. The register pair is 16-bit even though the CPU data path is 8-bit.

PC and DPTR are not interchangeable

PC serves instruction sequencing. DPTR provides an address for data/table operations, such as an external-RAM access or a program-memory lookup. Neither “8-bit microcontroller” nor “8-bit data bus” limits the address space to 256 bytes.

Correction to slide 53: many operations use A, but results are not always stored only in A. MUL AB produces a result in B:A, DIV AB changes both A and B, and some instructions operate on other registers or RAM directly.

9 · PSW: flags and bank selection

Lecture slides 53, 61 and 67; supplementary flag explanations [R6-R7]



Figure 5. Program Status Word. Bit 1 has no named arithmetic-status function in the basic teaching layout; derivatives may name it differently.

Bit	Name	What it means
7	CY	Carry from bit 7 in addition; borrow indication in subtraction.
6	AC	Carry from bit 3 to bit 4 in addition; useful for BCD arithmetic.
5	F0	User-controlled software flag.
4, 3	RS1, RS0	Select bank 0, 1, 2 or 3: 00, 01, 10 or 11.
2	OV	Signed overflow indication for addition/subtraction.
1	-	Spare/user-definable bit; no arithmetic-status role.
0	P	1 if A has an odd number of 1 bits; 0 if it has an even number.

Carry and signed overflow answer different questions

For **unsigned** 8-bit addition, CY indicates that the mathematical sum exceeded 255. For **signed two's-complement** addition, OV indicates that the sum is outside -128 to +127. The stored result is the low 8 bits in either interpretation.

Addition	A result	CY	AC	OV	P
7FH + 01H	80H	0	1	1	1
FFH + 01H	00H	1	1	0	0
0FH + 01H	10H	0	1	0	1

Interpret the first row: 127+1 is 128, which fits unsigned 8-bit data, so CY=0. It does not fit signed 8-bit data and the stored 80H represents -128, so OV=1. The low nibble overflows, giving AC=1. 80H has one set bit, giving P=1.

Parity is about A

P reflects the bits currently in the accumulator. A=03H has two 1 bits, so P=0; A=07H has three, so P=1. The PSW is bit-addressable: PSW.4 is bit address D4H and PSW.3 is D3H.

10 · Internal RAM and SFR space

Lecture slides 58, 65 and 68-69; SFR access clarified by [R3-R4]

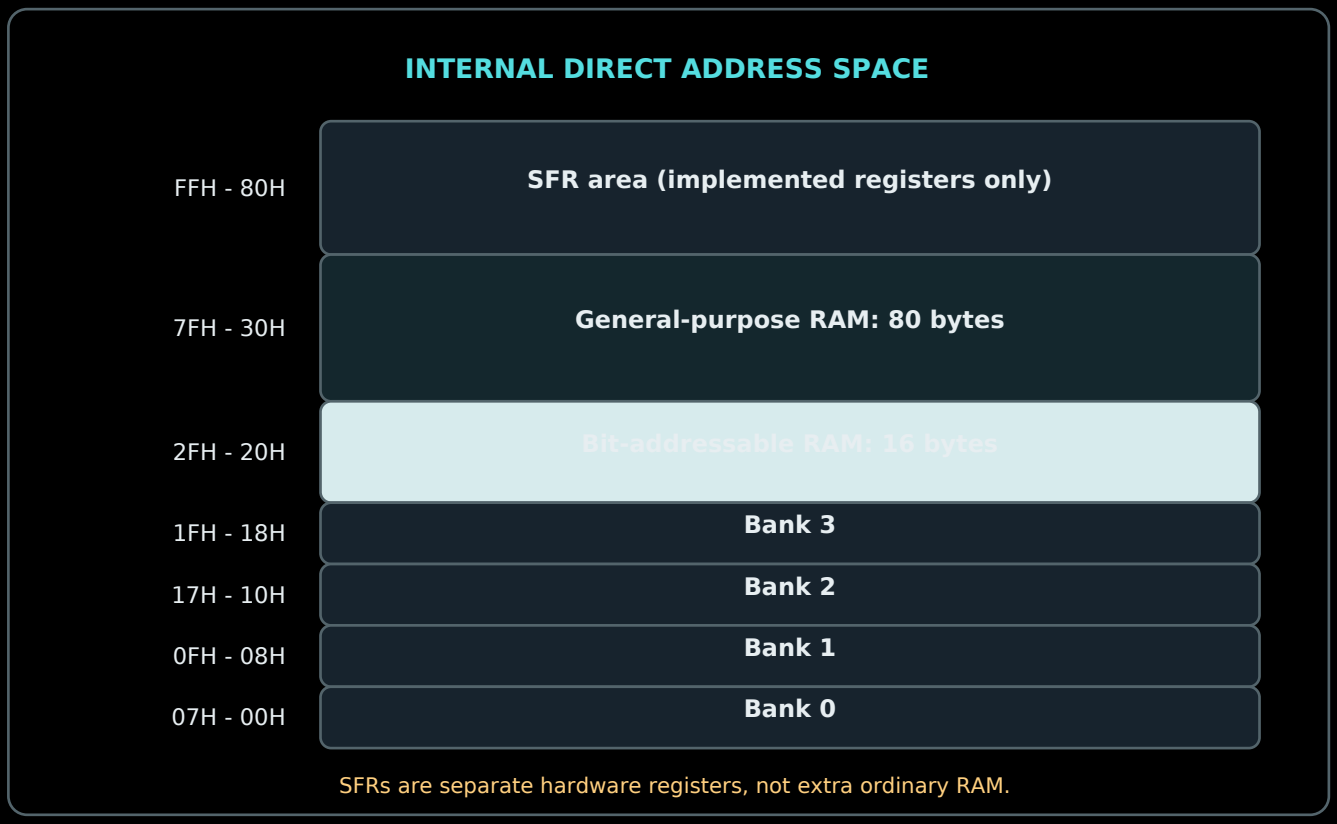


Figure 6. Direct-address view of the classic 8051. The lower 128 bytes are RAM; the upper direct-address region selects SFR hardware.

The lower 128 bytes: three useful regions

00H-1FH: 32 bytes containing four register banks, each with R0-R7. **20H-2FH:** 16 bytes whose individual bits are separately addressable. **30H-7FH:** 80 bytes normally used as general-purpose or scratchpad RAM.

Check the count: 32 + 16 + 80 = **128 bytes**. An inclusive address range has (last address - first address + 1) locations. Thus 7FH-30H+1 = 50H = 80 decimal.

The upper direct-address region is not another 128 bytes of RAM

80H-FFH is the SFR address region. Only defined registers exist; unused addresses must not be treated as storage. SFRs control the ports, timers, serial block, interrupts and CPU state. They are accessed directly, not through an ordinary indirect RAM pointer.

A variant distinction that prevents a common error

The classic 8051 has no upper 128-byte internal RAM block. The 8052 has additional RAM at indirect addresses 80H-FFH. In that device, **direct access selects an SFR** while **indirect access selects upper RAM**, despite the same numeric address.

The stack uses internal RAM and shares it with other allocated data. It is not a physically separate memory block permanently confined to 00H-1FH.

11 · Register banks and the stack

Lecture slides 58-61; supporting stack behavior [R5]

Four banks, eight registers per bank

RS1	RS0	Bank	R0 location	R7 location
0	0	0 (reset default)	00H	07H
0	1	1	08H	0FH
1	0	2	10H	17H
1	1	3	18H	1FH

The names R0-R7 refer to the **selected bank**; the other banks still retain their contents. A register's physical RAM address is **8 × bank number + register number**. For bank 2, R5 is at 2×8+5 = 21 decimal = 15H. For bank 3, R7 is at 1FH.

```
SETB PSW.4   ; RS1 = 1
CLR  PSW.3   ; RS0 = 0: bank 2 selected
```

Bank switching can give an interrupt routine a separate working-register set and reduce save/restore work. It does not automatically save A, B, DPTR or all other CPU state. Changing the entire PSW just to select a bank can also alter flags.

Stack behavior

The stack stores temporary saved data and return addresses in internal RAM. It follows **last in, first out**. SP resets to **07H**; a PUSH first increments SP, then stores the byte. A POP retrieves the top byte, then decrements SP. The stack therefore grows toward increasing addresses.

Action, starting with SP=2FH	SP after action	Stored / recovered data
PUSH ACC, with A=12H	30H	RAM[30H] = 12H
PUSH B, with B=34H	31H	RAM[31H] = 34H
POP B	30H	B receives 34H
POP ACC	2FH	A receives 12H

Why relocating the stack can matter

After reset, the first pushed byte goes to 08H, overlapping bank 1. Setting SP=2FH makes the first push use 30H. This is an example layout, not a universal rule: reserve enough free RAM above the chosen start and avoid variable/stack collisions.

12 · Bit-addressable RAM

Lecture slides 62-64, including the image-only mapping table

Bytes **20H-2FH** can be accessed either as 16 ordinary bytes or as **128 individual bits**. Their bit addresses are **00H-7FH**. These are two ways to refer to the same storage, not additional memory.

RAM byte	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0
20H	07	06	05	04	03	02	01	00
21H	0F	0E	0D	0C	0B	0A	09	08
22H	17	16	15	14	13	12	11	10
23H	1F	1E	1D	1C	1B	1A	19	18
24H	27	26	25	24	23	22	21	20
25H	2F	2E	2D	2C	2B	2A	29	28
26H	37	36	35	34	33	32	31	30
27H	3F	3E	3D	3C	3B	3A	39	38
28H	47	46	45	44	43	42	41	40
29H	4F	4E	4D	4C	4B	4A	49	48
2AH	57	56	55	54	53	52	51	50
2BH	5F	5E	5D	5C	5B	5A	59	58
2CH	67	66	65	64	63	62	61	60
2DH	6F	6E	6D	6C	6B	6A	69	68
2EH	77	76	75	74	73	72	71	70
2FH	7F	7E	7D	7C	7B	7A	79	78

Table 1. Every bit address in the matrix is hexadecimal; bit 0 is the least significant bit. A useful mental anchor is byte 20H → bits 00H-07H.

Convert a bit address b to its RAM location

For 00H ≤ b ≤ 7FH:
Byte address = 20H + floor(b / 8)
Bit position = b mod 8
Reverse: **b = 8 × (byte address - 20H) + bit position**.

Example: bit address 42H is 66 decimal. 66 = 8×8+2, so it identifies bit 2 of RAM byte 28H. Conversely, bit 6 of byte 2AH has address 8×(2AH-20H)+6 = 56H.

Bit instructions such as SETB and CLR use a bit address. A byte-oriented MOV uses a byte address. The opcode and operand form determine which interpretation applies.

13 · Worked bit-address problems

Lecture slide 64 reconstructed from its image; additional examples created for practice

Rework all six examples from the slide

Instruction	Numeric bit address	Divide by 8	RAM byte / bit
SETB 42H	66 decimal	66 = 8×8 + 2	28H, bit 2
CLR 67H	103 decimal	103 = 8×12 + 7	2CH, bit 7
CLR 0FH	15 decimal	15 = 8×1 + 7	21H, bit 7
SETB 28H	40 decimal	40 = 8×5 + 0	25H, bit 0
CLR 12	12 decimal	12 = 8×1 + 4	21H, bit 4
SETB 05	5 decimal	5 = 8×0 + 5	20H, bit 5

The number-base trap in the original slide

The last two operands are written without H. Under the decimal convention used in the slide, **CLR 12** uses bit address 0CH and changes byte 21H bit 4. **CLR 12H** instead uses bit address 18 decimal and changes byte 22H bit 2. Always state the base explicitly in your working.

Byte address 28H is not bit address 28H

```
MOV 28H, #01H   ; writes the whole RAM byte 28H
SETB 28H        ; sets bit 0 of RAM byte 25H
```

The identical numeral selects different storage because the instructions use different address types. In contrast, **SETB 42H** and **SETB 28H.2**, where supported as a byte.bit notation by the assembler, describe the same bit in byte 28H.

Reverse example: identify the bit address

Find the bit address of byte 2DH bit 6:
2DH-20H = 0DH = 13 decimal.
b = 13×8+6 = 110 decimal = **6EH**.

Try these before checking

1. Which byte/bit does CLR 5BH modify?
2. What is the bit address of byte 24H bit 3?
3. If byte 20H initially contains 00H, what is its value after SETB 00H and SETB 07H?

Answers: 1. Byte 2BH, bit 3. 2. 23H. 3. 81H. SETB sets one bit to 1; CLR clears one bit to 0 and leaves the other bits unchanged.

14 · Special Function Registers

Lecture slides 65-67; full image-only SFR table reconstructed and corrected

An **SFR** is a hardware register with a defined control, status or data function. SFR byte addresses lie in **80H-FFH**. The table below lists the 21 named SFR bytes in the basic slide set; reserved locations are not ordinary RAM.

SFR	Address	Bit access?	Purpose
P0	80H	Yes	Port 0 latch / pin interface
SP	81H	No	Stack pointer
DPL	82H	No	DPTR low byte
DPH	83H	No	DPTR high byte
PCON	87H	No	Power / serial-related control; details vary by device
TCON	88H	Yes	Timer and external-interrupt control / status
TMOD	89H	No	Timer/counter mode selection
TL0	8AH	No	Timer 0 low byte
TL1	8BH	No	Timer 1 low byte
TH0	8CH	No	Timer 0 high byte
TH1	8DH	No	Timer 1 high byte
P1	90H	Yes	Port 1 latch / pin interface
SCON	98H	Yes	Serial control / status
SBUF	99H	No	Serial data buffer
P2	A0H	Yes	Port 2 latch / pin interface
IE	A8H	Yes	Interrupt enable control
P3	B0H	Yes	Port 3 latch / pin interface
IP	B8H	Yes	Interrupt priority control
PSW	D0H	Yes	CPU flags and register-bank selection
ACC	E0H	Yes	Accumulator
B	F0H	Yes	Multiply/divide and temporary register

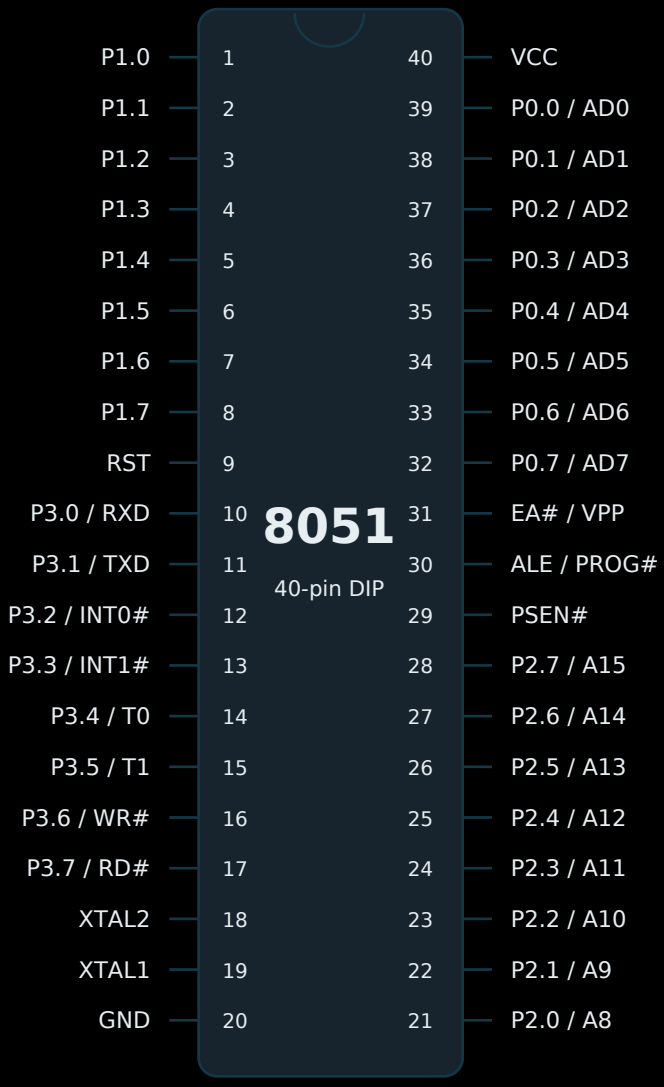
Which SFRs are bit-addressable?

Implemented SFRs whose byte address ends in **0H or 8H** support bit access. A bit's address is the SFR byte address + bit position. Example: P1.3 = 93H; PSW.7 (CY) = D7H. Reserved bits must still be respected. [R8]

Important correction: slide 66 prints DPL as 821H; the correct address is 82H. Also, the phrase "SFRs at the top of RAM" means the upper direct-address region, not an extra general-purpose RAM block. SBUF represents distinct receive/read and transmit/write buffers sharing one address.

15 · 8051 pin diagram

Lecture slides 70-75 • Classic 40-pin DIP package



denotes an active-low signal. Top view; notch at top.

Figure 7. Pin assignment redrawn for legibility. Programming-mode labels are retained where they appear in the slide pinout.

Pin group	Pins	Meaning
Supply	40 / 20	VCC / GND; classic teaching device uses +5 V
Clock	19 / 18	XTAL1 / XTAL2
Reset	9	RST: active-high reset input
Program-memory control	31 / 29 / 30	EA# / PSEN# / ALE
Parallel I/O	1-8, 10-17, 21-28, 32-39	P1, P3, P2 and P0 respectively

Use the top-view diagram carefully: pins run down the left from 1 to 20 and up the right from 21 to 40. Alternate functions share physical pins with port bits; enabling external memory or a peripheral reduces the pins available for ordinary GPIO.

16 · I/O port structure

Lecture slides 74-75; electrical behavior supplements the slides using [R1, R9]

Each of the four 8-bit ports has a latch, output-driver circuitry and an input path. Writing a port SFR changes its latch; reading the input path samples the pin. The basic 8051 does not use a separate direction register for these ports.

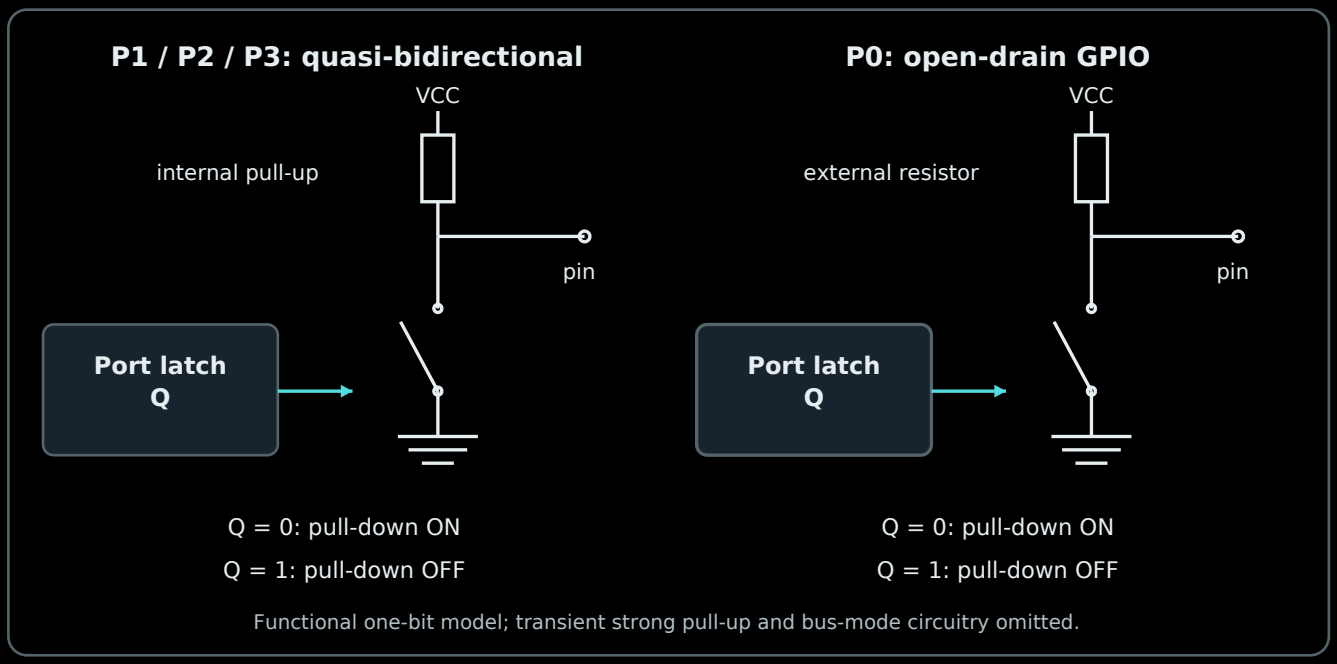


Figure 8. Simplified functional circuits for one GPIO bit. Q is the software-written latch value; this is not a complete transistor-level schematic.

Ports 1, 2 and 3: quasi-bidirectional

These ports have internal pull-ups. A latch value of 0 turns on the pull-down and drives the pin low. A latch value of 1 releases the strong pull-down; the pull-up holds the pin high, but an external device can pull it low. Therefore, write 1 to a bit before using it as an input.

Port 0: open-drain when used as GPIO

P0 has no internal GPIO pull-up. Writing 0 drives low; writing 1 releases the pin into a high-impedance state. An external pull-up is needed to obtain a defined high output level in normal GPIO use. During external-memory bus operation, P0 uses different driver control.

Latch Q	P1/P2/P3 as GPIO	P0 as GPIO
0	Actively pulls the pin low	Actively pulls the pin low
1	Pull-up provides high; external low can be read	Pin is released; high requires an external source/pull-up

Explain “quasi-bidirectional” in one sentence

The same port bit can act as an output or input without a separate direction register: a written 0 drives low, while a written 1 permits an external signal to determine the input state.

17 · Port roles and alternate functions

Lecture slides 70 and 73-75; read-versus-latch distinction [R10]

Port	Ordinary role	Additional hardware role
P0	8 GPIO bits; external pull-ups for high output	AD0-AD7: low address multiplexed with data for external-memory transfers
P1	8 quasi-bidirectional GPIO bits	No alternate peripheral functions in the basic 8051; derivatives differ
P2	8 quasi-bidirectional GPIO bits	A8-A15: high address for external code / 16-bit-address data accesses
P3	8 quasi-bidirectional GPIO bits	Serial, external interrupts, counter inputs and external data strobes

Port 3 bit	Pin	Alternate function
P3.0	10	RXD: serial receive
P3.1	11	TXD: serial transmit
P3.2	12	INT0#: external interrupt 0
P3.3	13	INT1#: external interrupt 1
P3.4	14	T0: Timer 0 external count input
P3.5	15	T1: Timer 1 external count input
P3.6	16	WR#: external-data-memory write strobe
P3.7	17	RD#: external-data-memory read strobe

Example: make P1.0 an input and read it

```
SETB P1.0    ; release the pull-down
MOV  C, P1.0 ; sample the pin into carry
```

If an external switch pulls P1.0 low, the read gives C=0 even though the port latch holds 1. If the pin remains high, the read gives C=1. Writing 0 would force the pin low and defeat this input arrangement.

Reading a pin is not always reading the latch

Ordinary input reads observe the pin. Port read-modify-write operations, such as ANL P1,#mask, use the latch value before updating it. Keep this distinction when explaining how output states are preserved. For a Port 3 alternate function, leave its latch bit at 1.

18 · External memory, clock and reset

Lecture slides 51 and 71-74; reset and clock timing [R9, R11]

Why address and data share P0

A 16-bit address and an 8-bit data value need many pins. The 8051 saves pins by putting the low address byte on P0 first, then reusing those same lines for data. **ALE** lets an external latch hold the low address while P0 changes function.

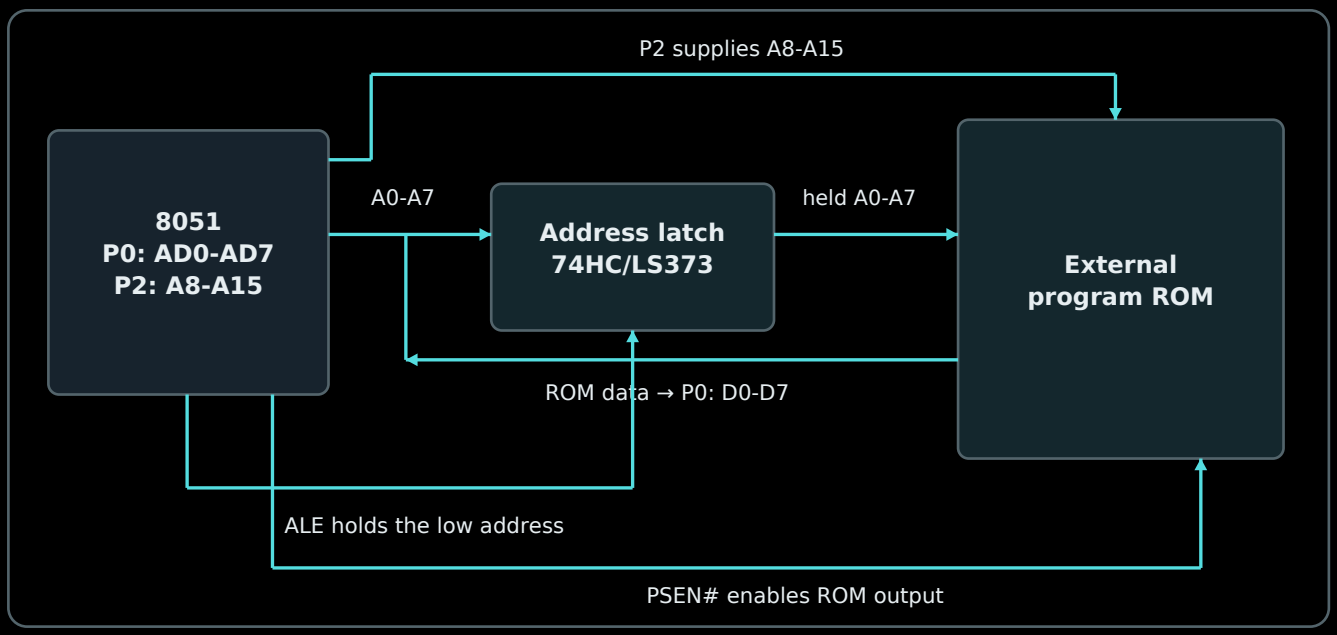


Figure 9. External program-memory interface, simplified. The address latch separates P0's address and data phases; external RAM uses RD#/WR# instead of PSEN#.

Signal	Meaning
ALE, pin 30	Address Latch Enable: captures/holds the low address from P0.
PSEN#, pin 29	Program Store Enable: active-low read strobe for external code.
EA#, pin 31	Low: all code external. High: internal code at 0000H-0FFFH on the 4 KB 8051; higher addresses use external code.
RD# / WR#	P3.7 / P3.6: active-low read/write controls for external data.

Clock and reset

XTAL1 and XTAL2 support the oscillator connection. The classic core uses **12 oscillator periods per machine cycle**: $T_{MC} = 12/f_{osc}$. At 12 MHz, $T_{MC} = 1$ µs. An instruction can take more than one machine cycle; newer cores may use different timing.

RST is active high and must remain high for at least **two machine cycles while the oscillator is running**. Reset starts PC at 0000H, SP at 07H and bank selection at bank 0; port latches become FFH. Ordinary RAM contents are not guaranteed to be cleared by reset.

EA# is tied high when using available internal program memory; high does not prohibit external code above its implemented range. The 8031 has no internal program ROM, so its code is fetched externally.

19 · Short-answer practice

Original practice based on the Module 1 syllabus; these are not claimed past-paper questions

The slide deck describes 3-mark Part A questions and 9-mark Part B questions. Use the compact answers below as recall prompts, not as a guaranteed marking scheme.

1. Define an embedded system and give two examples.

An embedded system combines hardware and software to perform a defined application, usually inside a larger product, under constraints such as power, cost, memory and response time. Examples: a washing-machine controller and an automotive ABS controller.

2. Distinguish hard and soft real-time systems.

Both have timing requirements. In hard real-time, missing a required deadline violates system correctness and may cause a serious failure; safety-critical flight control is an example. In soft real-time, occasional delays degrade quality while the result can remain useful; multimedia playback is an example.

3. State the major specifications of the classic 8051.

8-bit CPU; 4 KB on-chip program ROM; 128-byte internal RAM; four 8-bit I/O ports; two 16-bit timers/counters; one serial port; 16-bit PC and DPTR; separate 64 KB code and external-data spaces. Give the subset relevant to the question.

4. How does the 8051 select a register bank?

PSW bits RS1=PSW.4 and RS0=PSW.3 select one of four R0-R7 banks. 00 selects bank 0, 01 bank 1, 10 bank 2 and 11 bank 3. Bank 0 is selected after reset.

5. What is bit-addressable RAM?

The 16 bytes at 20H-2FH support individual-bit access, giving 128 bit addresses from 00H to 7FH. A bit address b identifies byte 20H+floor(b/8) and bit position b mod 8.

6. Define an SFR and give examples.

An SFR is a hardware register for CPU or peripheral control, status or data. Its direct byte address lies in 80H-FFH. Examples: ACC at E0H, PSW at D0H, P1 at 90H and SP at 81H. Only implemented SFR locations are valid.

7. Why does Port 0 need a pull-up in GPIO mode?

Its output is open-drain: 0 drives low, but 1 releases the pin. An external pull-up provides a defined high level. Ports 1, 2 and 3 have internal pull-ups.

8. Explain ALE and PSEN#.

ALE controls a latch that holds the low address emitted on P0 before those lines are reused for data. PSEN# is the active-low external-program-memory read strobe. External-data memory instead uses RD# and WR#.

20 · Build strong long answers

Original exam-preparation outlines aligned to the syllabus

A. Explain embedded systems and their characteristics.

Start: define a dedicated hardware-software system. **Draw:** Figure 1 with sensor, input interface, controller, output driver and actuator; show feedback. **Explain:** the task of each block with a thermostat or washing-machine example. **Develop:** dedicated function, resource constraints, reactive behavior, timing and reliability. **Finish:** list several application domains and distinguish embedded design from general-purpose computing.

B. Explain the architecture of the 8051.

Start: identify an 8-bit Harvard microcontroller and state its key resources. **Draw:** Figure 4. **Explain:** CPU/ALU, A and B, PSW, PC/DPTR, program ROM, internal RAM, ports, timers/counters, UART, interrupts, oscillator and control. **Connect:** instruction fetch/decode/execute and the movement of data. Keep the basic 8051 separate from enhanced derivatives.

C. Explain 8051 memory and register organization.

Draw: Figure 6 and label every boundary. **Calculate:** 32-byte bank region + 16-byte bit RAM + 80-byte scratchpad = 128 bytes. **Explain:** RS1/RS0 bank selection, the stack’s shared RAM use, bit mapping and the SFR direct-address region. **Work one example:** bank 2 R5 → 15H, or bit 42H → byte 28H bit 2. Mention separate code and external-data spaces.

D. Explain I/O ports and their structure.

Draw: the simplified one-bit circuits in Figure 8. **Compare:** P0’s open-drain GPIO with P1/P2/P3’s internal pull-ups. **Explain:** latch 0 drives low; latch 1 enables input behavior. **List:** P0/P2 bus functions and all eight P3 alternate functions. **Show:** why pin reads and read-modify-write latch reads differ.

What gives an answer substance

Use a labeled diagram, accurate address ranges, a clear explanation of behavior, and a worked example. A page of disconnected feature names is weaker than a short explanation connecting each block to its purpose.

Draw-from-memory checklist

1. Embedded feedback loop.
2. 8051 functional architecture.
3. PSW bit layout and register-bank table.
4. Internal RAM/SFR map with 00H, 20H, 30H, 80H boundaries.
5. Port circuits, P3 alternate functions and the external-memory latch.

21 · Self-test and worked solutions

Original questions for recall and numerical practice

Question	Answer / essential working
1. Bank 3 is selected. What RAM address holds R4?	3×8+4 = 28 decimal = **1CH**.
2. RS1=0 and RS0=1. Which bank and address for R6?	Bank 1; 8+6 = 14 decimal = **0EH**.
3. Decode bit address 76H.	118 = 8×14+6; byte 20H+0EH = **2EH, bit 6**.
4. Find the bit address of byte 27H bit 5.	8×7+5 = 61 decimal = **3DH**.
5. How many bytes are in 30H-7FH?	7FH-30H+1 = 50H = **80 bytes**.
6. SP=3FH. PUSH ACC; PUSH B; POP B. Final SP?	40H → 41H → **40H**; ACC remains saved at 40H.
7. DPTR=9B40H. What are DPH and DPL?	**DPH=9BH; DPL=40H**.
8. A=81H. What is the parity flag?	Two set bits, so **P=0**.
9. 7FH+01H: why CY=0 but OV=1?	128 fits unsigned 8-bit data, but exceeds signed +127.
10. What does bit address 93H refer to?	SFR-bit space: P1 at 90H; **P1.3**.
11. P1 latch bit is 1 but an external input pulls the pin low. What does a pin read see?	**0**. The written 1 releases the pull-down so the external source can pull low.
12. A 12 MHz classic 8051 requires a 2-machine-cycle reset pulse. Minimum duration?	12/12 MHz = 1 µs per machine cycle; **2 µs**, after the oscillator is running.

Scenario: choose and justify

A battery sensor measures once per second, stores a small result and sleeps between measurements. What matters most? **Answer:** sufficient processing and RAM/program memory, the required sensor interface, low sleep/active energy, suitable wake-up timing, cost, package and development support. If the sensor is analog, the basic 8051 needs an external ADC.

Scenario: identify the real-time requirement

A computed sample is unusable after the next sampling interval, but occasional discarded samples are allowed. **Answer:** firm real-time for that sample. If the late result remained useful but reduced quality, the requirement would be soft real-time; if any miss violated a strict correctness requirement, it would be hard real-time.

22 · Final revision sheet

Use after studying; corrections below prevent common slide-derived mistakes

Remember	Value / rule
Classic on-chip resources	4 KB program ROM; 128 B RAM; 32 I/O; 2 timers; 1 UART
Register widths	A/B/PSW/SP/Rn = 8 bits; PC/DPTR = 16 bits
RAM allocation	00H-1FH banks; 20H-2FH bit RAM; 30H-7FH scratchpad
Bank selection	RS1:RS0 = 00, 01, 10, 11 → banks 0, 1, 2, 3
Bank register address	8 × bank + register number
RAM bit mapping	Byte=20H+floor(bit address/8); position=bit address mod 8
SFR bit mapping	For a valid bit-addressable SFR: byte address + bit position
Address anchors	P0=80H; SP=81H; DPL=82H; DPH=83H; PSW=D0H; ACC=E0H; B=F0H
Input rule	Write 1 first; P0 needs an external GPIO pull-up
Bus pins	P0=AD0-AD7; P2=A8-A15; ALE=latch; PSEN#=code read
Reset / timing	RST active high; SP=07H; classic machine cycle=12/fosc

Six misconceptions to avoid

• **“The 8051 has 256 bytes of RAM because the map ends at FFH.”** It has 128 bytes; the upper direct-address region is SFR space.

• **“The stack is permanently limited to the register-bank region.”** It uses allocated internal RAM and can be relocated; prevent overlap with banks and variables.

• **“DPL is at 821H.”** This is a slide typo. DPL is at 82H.

• **“Reset clears all RAM and every register becomes zero.”** Reset initializes control state; SP=07H and ports=FFH are counterexamples. RAM is not guaranteed cleared.

• **“The 8051 has an ADC because microcontrollers often have ADCs.”** The classic 8051 does not; derivatives may.

• **“Six interrupts means six different vectors.”** The classic structure has six request sources but five vectors; serial transmit and receive share one.

Also keep CY separate from OV, bit addresses separate from byte addresses, and a 64 KB address space separate from the amount of physical on-chip memory. Detailed timer, serial, interrupt and interfacing programs are beyond this module.

23 · Sources and coverage record

Preparation date: 7 October 2026

Primary teaching source

Dr. Mayadevi N, CET: Module1-ES+8051.pdf, 75 pages, in the Microcontroller folder. The syllabus is read from slide 4's image table and slide 33's text summary. The course code shown on slide 2 is PCEOT501. Scope is the supplied slide syllabus; a separate university syllabus document was not present in that folder.

Syllabus item	Slide evidence	Note pages
Embedded systems, applications and characteristics	13-25, 35-37	2-4
Hard / soft real-time	26-30	5
Microprocessors vs microcontrollers	36-40, 43	6
8051 architecture and registers	44-57; images 48-49	7-10
RAM, register banks and bit RAM	58-64, 68-69	11-14
SFRs	65-67	15
I/O ports and pin functions	70-75	16-19

Supplementary technical references

[R1] **Intel, MCS-51 Microcontroller Family User's Manual.** Port pull-ups, latch/pin behavior and interrupt terminology; archived manufacturer manual.

[R2] **Arm/Keil, 8051 Variants.** Register-bank and programmer-visible register organization.

[R3] **Keil, Memory Space Utilization in C51, APNT_101.** Separate memory spaces and upper-RAM/direct-SFR distinction.

[R4] **Arm/Keil, External SFR Access for 8051 Programs.** SFRs require direct access.

[R5] **TEMIC, C51 Family Programming Guide.** Stack push/return operation.

[R6] **Atmel, 8051 MCU Instruction Set.** Carry, auxiliary carry and signed-overflow behavior.

[R7] **Hynix, HMS9xC8032 User's Manual.** Compatible PSW layout, parity and serial-buffer explanation.

[R8] **Silicon Labs, C8051F38x Data Sheet, SFR section.** SFR byte/bit address rule; device-specific additions excluded.

[R9] **Atmel/Microchip, AT89S51 Data Sheet.** Compatible pin/port and reset behavior; added AT89S51 features excluded.

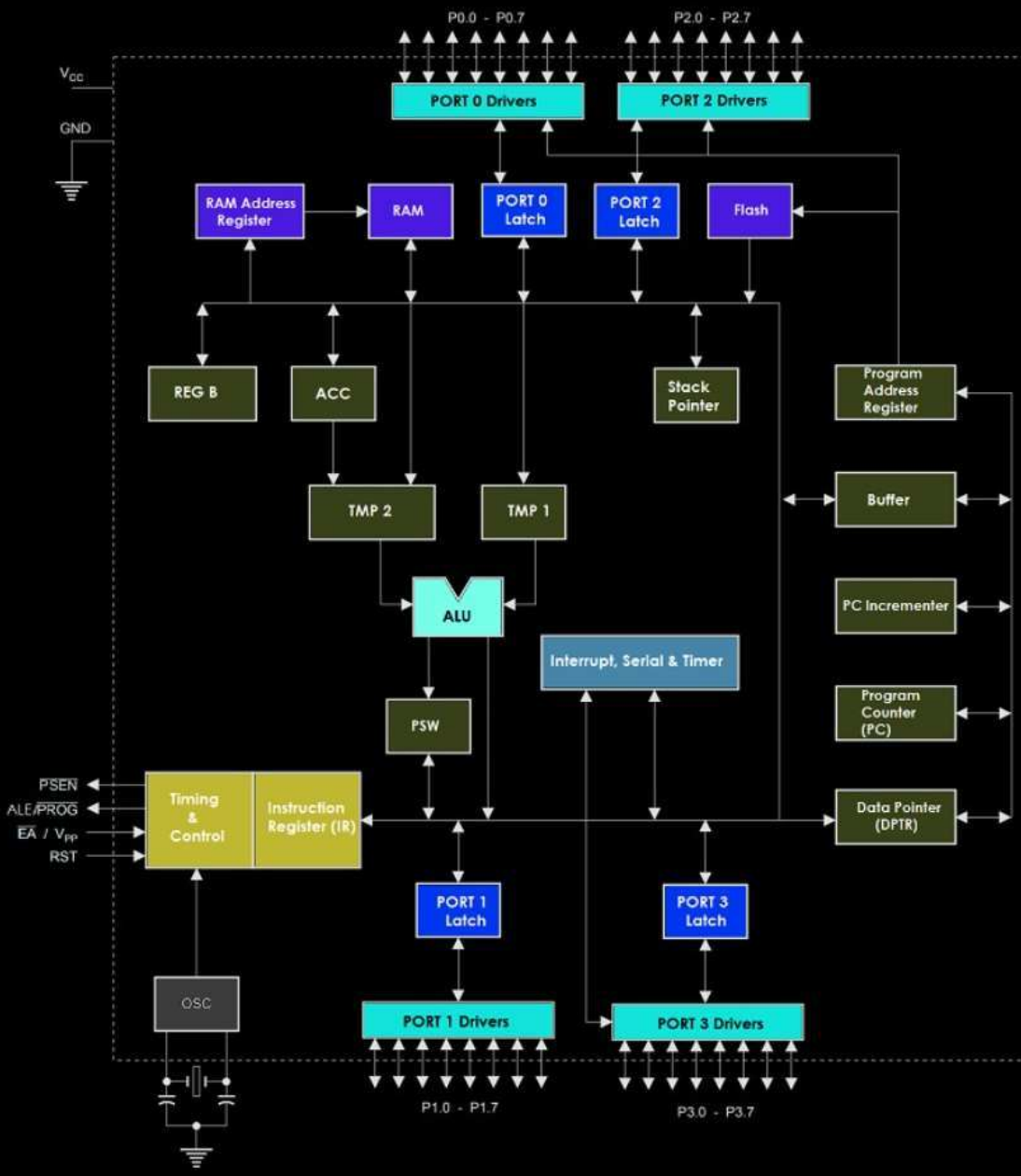
[R10] **Arm/Keil, How Do I Read a Latch?.** Port read-modify-write instructions use the latch value.

[R11] **Arm/Keil, 8051 Instruction Set Manual: Instructions.** Classic timing convention: 12 oscillator periods per machine cycle.

All diagrams in these notes were newly drawn. Worked practice and model-answer outlines were created for this guide; they are not presented as past questions or official answer keys. Numbered slide citations refer to PDF page numbers, counting the title slide as page 1.

Appendix A · Full internal 8051 architecture

Faithful reproduction of the lecture’s image-only architecture on slide 48, included at readable size; explanatory notes clarify the functional blocks.



Read the diagram as two connected paths. The datapath moves bytes among the RAM, port latches, accumulator, B register, temporary registers, ALU and PSW over the internal bus. The program path uses the PC, program address register, incrementer and buffer to fetch instructions; timing/control and the instruction register coordinate execution.

Appendix A continued · Block-by-block reading

Use this key alongside the complete architecture figure on the preceding page.

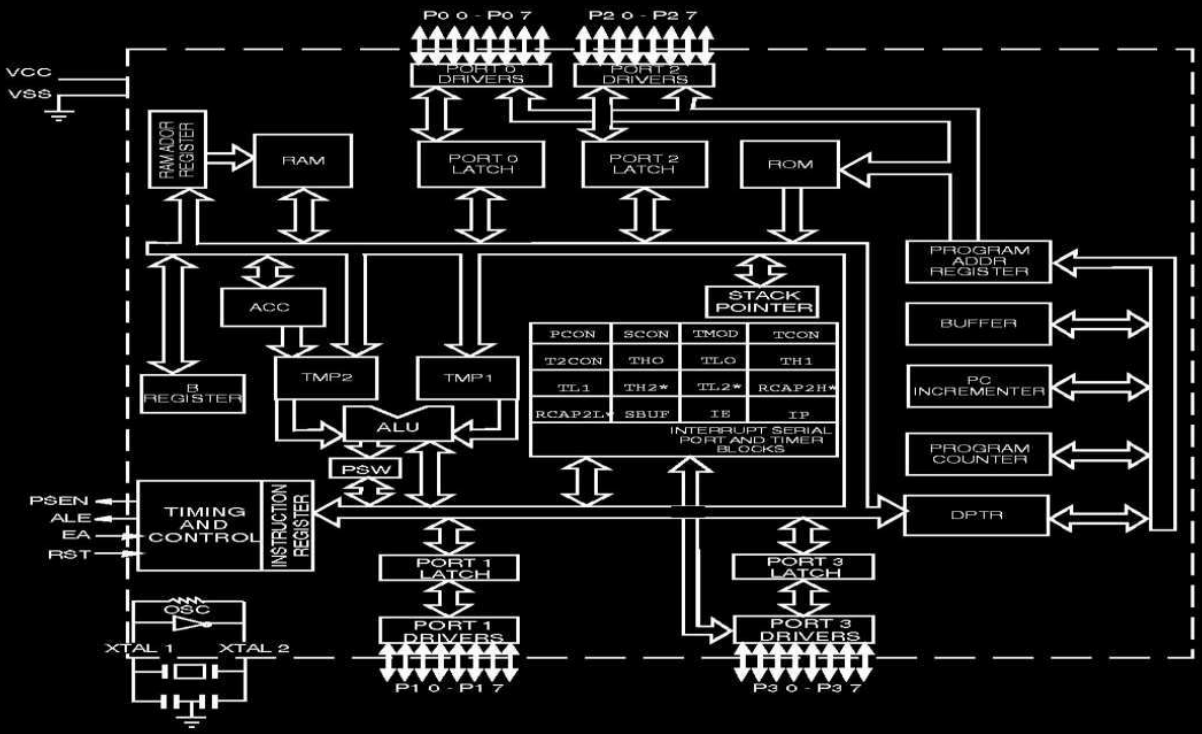
Block shown in the slide	What it contributes
RAM address register + RAM	Selects internal data RAM locations; classic 8051 implements 128 bytes.
ACC, B, TMP1/TMP2, ALU, PSW	Performs arithmetic/logic and holds the result and status flags.
PC, incrementer, program address register, buffer	Sequences instruction fetches from program memory; PC and DPTR are distinct 16-bit registers.
Port latches and drivers P0-P3	Connects the internal bus and port latches to the external pins; each port has different electrical / alternate-function behavior.
Interrupt, serial and timer block	Collects peripheral status/control registers and request logic; the basic 8051 has timers 0/1, serial I/O and interrupt sources.
Oscillator, timing/control, instruction register	Supplies timing, decodes/coordinates instructions and controls external memory signals.

Diagram correction for careful readers

The slide labels the internal program store “Flash.” The classic 8051 is conventionally specified with 4 KB on-chip program ROM; exact nonvolatile-memory technology depends on the compatible device. Use the named block’s function, and use the specific chip data sheet for its memory technology.

Appendix B · Family diagram and variant boundaries

Lecture slide 49 contains a second architecture image. It shows an enhanced 8051-family implementation, including Timer 2 registers, rather than the classic 8051 block set.



What to notice: the interrupt/serial/timer register block explicitly includes T2CON, TH2, TL2, RCAP2H and RCAP2L. Those are Timer 2 control/count/reload registers found in 8052-class derivatives. The dashed device boundary and shared 8051-compatible CPU organization explain why the overall diagram resembles slide 48.

Appendix B continued · Keep the variants straight

The family diagram is reproduced on the preceding page; use this comparison to separate classic 8051 facts from derivative features.

Classic 8051 (syllabus baseline)	8052-class enhancement shown in slide 49
128 bytes internal RAM	256 bytes internal RAM on the classic 8052, with upper RAM indirectly addressed
Two 16-bit timers/counters: Timer 0 and Timer 1	Adds Timer 2, including capture/reload registers
4 KB internal program ROM in the canonical part	Program-memory capacity/type varies by derivative
No T2CON, TH2, TL2 or RCAP2 SFRs	Timer 2 SFRs appear in the family diagram

Exam rule

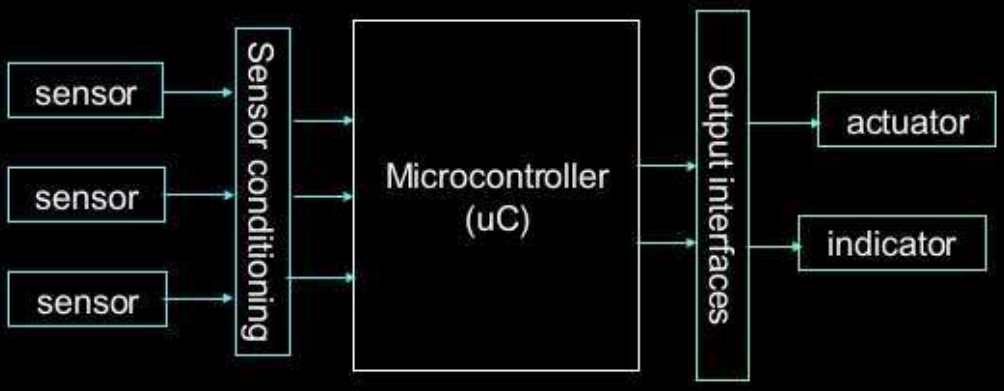
For questions that say simply “8051,” draw the classic architecture from Appendix A and name timers 0 and 1. Mention Timer 2 only when the question specifies 8052 or a compatible derivative that implements it.

Family comparisons on slides 44-47 introduce 8031, 8051 and 8052 variants from multiple manufacturers. Exact ROM/EPROM/Flash size, RAM and peripheral details depend on the named device; do not transfer a derivative feature to the classic 8051.

Appendix C · Source diagrams: system model and comparison

Original lecture figures from slides 35 and 39 are retained as images so the slide-based system model and block comparison are both represented.

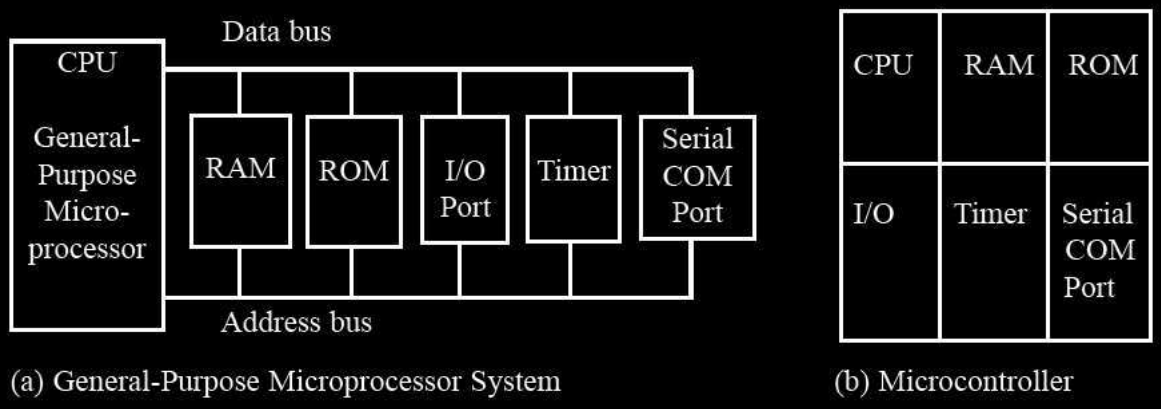
Slide 35: embedded-system model. The lecture figure shows multiple sensors feeding signal conditioning, then a microcontroller, output interface and actuator, with an indicator.



Appendix C continued · Processor and controller blocks

Original figure from lecture slide 39. The vector comparison table and the systems figure on page 28 complement this source image.

Slide 39: microprocessor versus microcontroller block comparison. The key distinction is integration: a microprocessor system typically places memory and I/O outside the CPU; a microcontroller integrates CPU, memory and peripherals.



Other slide figures represented in the main notes

The 4-panel result-value comparison (non-real-time, hard, firm and soft) is redrawn on page 5; the 40-pin package on page 16, port bit circuits on page 17, external-memory latch on page 19, memory/register maps and full bit table on pages 12-15 are clean vector redraws. The source 8051 and 8052-family architecture images are retained on pages 25-26.

Appendix D · Full slide and syllabus coverage

Every source slide is assigned below. “Outside Module 1” identifies course administration or material outside the stated nine-hour Module 1 syllabus; it is recorded for completeness, not expanded as examinable Module 1 content.

Lecture slides	Content checked	Coverage in these notes
1-4	Course title, course objectives and image-only syllabus tables	Course context below; Module 1 syllabus detailed on pages 1 and 24.
5-12	Module contents, assessment, course outcomes and textbooks	Course context below; ARM programming objective is outside Module 1.
13-17	Embedded-system definition, dedicated purpose and application domains	Pages 2-3.
18-22	Application list, automotive figure, characteristics, design criteria/challenges and trends	Pages 3-4 and C.
23-25	Embedded versus general computing; price-power-performance tradeoff	Pages 4 and 6.
26-30	Real-time definition, deadline/value concepts, hard and soft real-time illustrations	Page 5; firm real-time is identified as a useful supporting distinction.
31-35	Microcontroller transition and system model figure	Appendix C; controller feedback loop on page 2.
36-43	Microprocessor/microcontroller comparison and embedded-control context	Page 6 and Appendix C.
44-50	8051-compatible family, specifications and architecture figures	Pages 7-10 and Appendices A-B; slide 49 is identified as 8052-class.
51-57	Program/data spaces, external-memory organization and programmer-visible registers	Pages 10, 19 and 21-22.
58-64	RAM map, register banks, PSW and image-only bit mapping/examples	Pages 11-14.
65-69	SFR addresses, bit-addressable SFR table and complete memory map	Pages 12, 14-15 and 23.
70-75	Pinout, alternate functions, port structure and signal behavior	Pages 16-19; pinout and port circuits redrawn.

Appendix D continued · Course context and scope

Opening-slide details and the exact Module 1 syllabus text are recorded here, including material outside the module boundary.

Course and assessment context from the opening slides

The deck identifies **PCEOT501, Microcontrollers and Embedded Systems**, with CIE worth 40 marks and ESE worth 60 marks (slides 2 and 9-10). It lists two course objectives: foundations in 8051 principles/programming/applications and ARM architecture/programming. The second objective and ARM material are broader course scope, not part of this Module 1 note.

The Module 1 syllabus is listed on slide 4 as an image and restated on slide 33: **9 hours** covering introduction to microprocessors and microcontrollers; comparison; embedded applications, features and characteristics; hard and soft real-time systems; and 8051 architecture, I/O port structure, register organization, general-purpose RAM, bit-addressable RAM and SFRs. The note follows this stated scope and includes firm real-time only as a helpful distinction.

The opening slides also include module learning outcomes and recommended textbooks. Their wording is course framing rather than additional 8051 subtopics; it is recorded here so the note does not imply that the lecture contained only technical slides.

Completeness method: the full 75-page PDF was text-extracted and image-only syllabus, architecture, memory/register maps, examples and pin/port figures were inspected visually. Where slide wording conflicts with the classic device specification, this guide flags the correction instead of silently repeating the error.